SENAI REGIONAL METROPOLITANA
CAMAÇARI/BA

DISCIPLINA: Interação com API DOCENTE: VAGNER LIMA

Index.html

```html
<!-- API - Dev: Vagner Lima 03.11.2026 -->
<!DOCTYPE html>
<html lang="pt-br">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>API - Application Programming Interface</title>
    <link rel="stylesheet" href="style.css">
</head>

<body>

    <h2>Cadastrar Pokémon</h2>
    <form id="pokemonForm">
        <input type="text" id="nomeInput" placeholder="Nome" required>
        <input type="text" id="tipoInput" placeholder="Tipo" required>
        <button type="submit">Adicionar</button>
    </form>

    <h2>Lista da Minha API</h2>
    <ul id="listaPokemons"></ul>

    <script src="api.js"></script>
</body>

</html>
```

style.css

```css
/* Estilo API - Dev: Vagner Lima 03.11.2026 */
body {
    font-family: sans-serif;
    padding: 20px;
    max-width: 500px;
    margin: auto;
}
form {
    display: flex;
    gap: 8px;
    margin-bottom: 20px;
}
input {
    padding: 8px;
    flex: 1;
}
button {
    padding: 8px 12px;
    cursor: pointer;
}
ul {
    list-style: none;
    padding: 0;
}
li {
    background: #eee;
    margin: 5px 0;
    padding: 10px;
    display: flex;
    justify-content: space-between;
    align-items: center;
    border-radius: 4px;
}
```

api.js

```
/* API - Dev: Vagner Lima 03.11.2026 */

// 1. Guarda em uma constante a URL base da nossa API que está rodando no Node.js (porta 3000)
const API_URL = 'http://localhost:3000/api/pokemons';

// 2. Captura os elementos da página HTML usando os IDs para podermos interagir com eles via JavaScript
const form = document.getElementById('pokemonForm'); // O formulário de cadastro
const nomeInput = document.getElementById('nomeInput'); // O campo de texto do nome
const tipoInput = document.getElementById('tipoInput'); // O campo de texto do tipo
const listaPokemons = document.getElementById('listaPokemons'); // A lista <ul> onde os Pokémons serão inseridos

// --- OPERAÇÃO 1: READ (Ler / Listar) ---
// Função assíncrona que busca a lista de Pokémons na API e desenha os itens na tela
async function carregarPokemons() {
    // Faz uma requisição do tipo GET para o servidor e aguarda a resposta (resposta de rede)
    const res = await fetch(API_URL);

    // Converte os dados brutos recebidos da API de texto JSON para um Array em JavaScript
    const dados = await res.json();

    // Limpa o conteúdo atual da lista na tela para não acumular nem duplicar Pokémons antigos
    listaPokemons.innerHTML = '';

    // Percorre cada objeto 'pokemon' dentro do Array 'dados' recebido do backend
    dados.forEach(pokemon => {
        // Cria um novo elemento de lista HTML (<li>) na memória do navegador
        const li = document.createElement('li');

        // Define o HTML interno da <li> mostrando o ID, nome, tipo e criando o botão Excluir com o ID correto
        li.innerHTML = `
            <span><strong>#${pokemon.id} ${pokemon.nome}</strong> (${pokemon.tipo})</span>
            <button class="btn-del" onclick="deletarPokemon(${pokemon.id})">Excluir</button>
        `;

        // Adiciona a <li> recém-criada dentro da nossa lista <ul> que está visível no HTML
        listaPokemons.appendChild(li);
    });
}

// --- OPERAÇÃO 2: CREATE (Criar / Cadastrar) ---
// Adiciona um escutador de eventos para interceptar o momento em que o usuário envia (submit) o formulário
form.addEventListener('submit', async (e) => {
    // Previne o comportamento padrão do HTML, que seria recarregar a página inteira ao enviar o formulário
    e.preventDefault();

    // Cria um objeto JavaScript com os dados que o usuário digitou nos campos de texto (removendo espaços das pontas com .trim())
    const novoPokemon = {
        nome: nomeInput.value.trim(),
        tipo: tipoInput.value.trim()
    };

    // Faz uma requisição HTTP enviando os dados para a API
    const res = await fetch(API_URL, {
        method: 'POST', // Define o método HTTP como POST (usado para criação)
        headers: { 'Content-Type': 'application/json' }, // Avisa ao servidor que estamos enviando os dados no formato JSON
        body: JSON.stringify(novoPokemon) // Converte o objeto JavaScript em uma String/Texto formatado em JSON
    });

```

```
    // Se a API responder com sucesso (Status 200 até 299)
    if (res.ok) {
        nomeInput.value = ''; // Limpa o campo de texto do nome
        tipoInput.value = ''; // Limpa o campo de texto do tipo
        carregarPokemons();   // Recarrega a lista para buscar e exibir o novo Pokémon cadastrado
    }
});

// --- OPERAÇÃO 3: DELETE (Excluir / Remover) ---
// Função assíncrona que recebe o ID do Pokémon que deve ser apagado
async function deletarPokemon(id) {
    // Faz uma requisição chamando a URL com o ID específico no final (exemplo: http://localhost:3000/api/pokemons/1)
    const res = await fetch(`${API_URL}/${id}`, {
        method: 'DELETE' // Define o método HTTP como DELETE (usado para remoção)
    });

    // Se o servidor confirmar que excluiu com sucesso
    if (res.ok) {
        carregarPokemons(); // Recarrega a lista para remover o Pokémon apagado da tela
    }
}

// Executa a função imediatamente quando a página é carregada pela primeira vez no navegador
carregarPokemons();
```

server.js

```
/* API - Dev: Vagner Lima 03.11.2026 */

// Importa o módulo 'http' que é nativo do Node.js, usado para criar servidores web
const http = require('http');

// Cria uma variável (array) para simular nosso "Banco de dados" na memória RAM do servidor
// Se o servidor for reiniciado, esses dados voltam ao estado inicial
let pokemons = [
  { id: 1, nome: "Pikachu", tipo: "Elétrico" },
  { id: 2, nome: "Charmander", tipo: "Fogo" }
];

// --- FUNÇÕES AUXILIARES ---

// Cria uma função para padronizar e facilitar o envio de respostas JSON para o frontend
const enviarJSON = (res, statusCode, dados) => {
  // 'res.writeHead' configura o cabeçalho da resposta: diz qual é o código de status (ex: 200, 404)
  // e avisa ao navegador que o conteúdo enviado é do tipo JSON
  res.writeHead(statusCode, { 'Content-Type': 'application/json' });
  // 'res.end' finaliza a resposta enviando os dados convertidos de objeto JavaScript para texto (string JSON)
  res.end(JSON.stringify(dados));
};

// Função para ler o corpo (body) de requisições que enviam dados (como POST e PUT)
const receberBody = (req) => {
  // Retorna uma Promise, pois o recebimento de dados pela rede é um processo assíncrono (demora um tempo)
  return new Promise((resolve) => {
    let corpo = ''; // Variável para armazenar o texto que vai chegar

    // O evento 'data' é disparado toda vez que um "pedaço" (chunk) de informação chega pela rede
    // Nós pegamos esse pedaço, convertemos para texto e juntamos na variável 'corpo'
    req.on('data', chunk => { corpo += chunk.toString(); });
```

```
    // O evento 'end' avisa que a transmissão de dados acabou
    req.on('end', () => {
      // Se houver texto no corpo, converte de JSON (texto) para Objeto JavaScript e resolve a Promise
      // Se estiver vazio, devolve um objeto vazio {}
      resolve(corpo ? JSON.parse(corpo) : {});
    });
  });
};

// --- CRIAÇÃO DO SERVIDOR ---

// 'createServer' cria o servidor web. Toda vez que alguém acessar o servidor, essa função será executada
// Usamos 'async' porque vamos precisar esperar (await) a leitura do body em algumas rotas
const server = http.createServer(async (req, res) => {

  // --- MIDDLEWARE MANUAL: Configuração de CORS ---
  // Libera o acesso para qualquer frontend (qualquer URL) fazer requisições a esta API
  res.setHeader('Access-Control-Allow-Origin', '*');
  // Informa ao navegador quais métodos HTTP são permitidos no nosso servidor
  res.setHeader('Access-Control-Allow-Methods', 'GET, POST, PUT, DELETE, OPTIONS');
  // Permite que o frontend envie cabeçalhos específicos (neste caso, o Content-Type)
  res.setHeader('Access-Control-Allow-Headers', 'Content-Type');

  // Quando o frontend faz um POST/PUT/DELETE, o navegador manda antes uma requisição escondida chamada 'OPTIONS' (Preflight)
  // Aqui nós interceptamos o OPTIONS, dizemos "tudo certo" (status 204 - sem conteúdo) e encerramos
  if (req.method === 'OPTIONS') {
    res.writeHead(204);
    return res.end();
  }
```

```
// --- ROTAS DO CRUD ---

// Expressão Regular (Regex) para verificar se a URL digitada tem o formato '/api/pokemons/NUMERO'
// O '(\d+)' captura a parte numérica da URL (ex: captura o '1' de /api/pokemons/1)
const idMatch = req.url.match(/\/api\/pokemons\/(\d+)/);

// Se a URL bater com a Regex, extraímos o número e convertemos para Inteiro. Se não, id vira null.
const id = idMatch ? parseInt(idMatch[1]) : null;

try { // Bloco try/catch para capturar qualquer erro fatal e não deixar o servidor "cair"

  // 1. READ (GET) - Listar todos os Pokémons
  // Se a rota acessada for exatamente '/api/pokemons' e o método for GET
  if (req.url === '/api/pokemons' && req.method === 'GET') {
    // Devolve status 200 (Sucesso) e a lista inteira de pokémons
    return enviarJSON(res, 200, pokemons);
  }

  // 2. READ (GET por ID) - Buscar um Pokémon específico
  // Se achou um ID na URL e o método for GET
  if (idMatch && req.method === 'GET') {
    // Procura no array um pokémon que tenha o mesmo ID da URL
    const pokemon = pokemons.find(p => p.id === id);
    // Se não encontrou, devolve erro 404 (Não Encontrado)
    if (!pokemon) return enviarJSON(res, 404, { mensagem: "Pokémon não encontrado!" });
    // Se encontrou, devolve status 200 e os dados do pokémon específico
    return enviarJSON(res, 200, pokemon);
  }

```

```
  // 3. CREATE (POST) - Adicionar um novo Pokémon
  // Se a rota for '/api/pokemons' e o método for POST
  if (req.url === '/api/pokemons' && req.method === 'POST') {
    // Aguarda (await) a função ler todos os dados enviados e extrai 'nome' e 'tipo'
    const { nome, tipo } = await receberBody(req);

    // Validação: se o usuário não enviou o nome ou o tipo, devolve erro 400 (Requisição Inválida)
    if (!nome || !tipo) {
      return enviarJSON(res, 400, { mensagem: "Nome e Tipo são obrigatórios!" });
    }

    // Monta o objeto do novo pokémon
    const novoPokemon = {
      // Lógica para criar um ID único: pega o ID do último elemento do array e soma 1. Se o array estiver vazio, o ID é 1.
      id: pokemons.length > 0 ? pokemons[pokemons.length - 1].id + 1 : 1,
      nome, // Equivale a nome: nome
      tipo  // Equivale a tipo: tipo
    };

    // Adiciona o novo pokémon na nossa lista (array) em memória
    pokemons.push(novoPokemon);
    // Retorna status 201 (Criado) e mostra os dados do pokémon recém criado
    return enviarJSON(res, 201, novoPokemon);
  }

```

```
  // 4. UPDATE (PUT) - Atualizar um Pokémon existente
  // Se achou um ID na URL e o método for PUT
  if (idMatch && req.method === 'PUT') {
    // Recebe os novos dados enviados pelo frontend
    const { nome, tipo } = await receberBody(req);
    // Descobre a posição (índice) desse pokémon dentro do array
    const index = pokemons.findIndex(p => p.id === id);

    // Se retornou -1, é porque o pokémon com aquele ID não existe no array
    if (index === -1) return enviarJSON(res, 404, { mensagem: "Pokémon não encontrado!" });

    // Substitui o item na posição encontrada pelo novo objeto com os dados atualizados
    pokemons[index] = { id, nome, tipo };
    // Retorna sucesso (200) e envia de volta o pokémon alterado
    return enviarJSON(res, 200, { mensagem: "Pokémon atualizado com sucesso!", pokemon: pokemons[index] });
  }

  // 5. DELETE (DELETE) - Remover um Pokémon
  // Se achou um ID na URL e o método for DELETE
  if (idMatch && req.method === 'DELETE') {
    // Descobre a posição (índice) do pokémon no array
    const index = pokemons.findIndex(p => p.id === id);

    // Se não achou, retorna erro 404
    if (index === -1) return enviarJSON(res, 404, { mensagem: "Pokémon não encontrado!" });

```

```
      // O comando 'splice' remove itens de um array. Aqui pedimos para remover 1 item a partir da posição (index).
      // Ele retorna um array com os itens removidos, por isso guardamos em 'pokemonRemovido'
      const pokemonRemovido = pokemons.splice(index, 1);
      // Retorna sucesso (200) e informa qual pokémon foi apagado
      return enviarJSON(res, 200, { mensagem: "Pokémon deletado com sucesso!", pokemon: pokemonRemovido[0] });
    }

    // --- FALLBACK (Rota Padrão) ---
    // Se o código chegou até aqui, significa que a URL e o Método digitados não caíram em nenhum dos IFs acima.
    // Portanto, a rota não existe. Retorna 404.
    enviarJSON(res, 404, { mensagem: "Rota não encontrada" });

  } catch (erro) {
    // Se ocorrer algum erro inesperado de código, o 'catch' segura o erro e devolve status 500 (Erro no Servidor)
    enviarJSON(res, 500, { mensagem: "Erro interno no servidor" });
  }
});

// --- INICIALIZAÇÃO DO SERVIDOR ---

// Pede para o servidor criado escutar as requisições na porta 3000 do seu computador
server.listen(3000, () => {
  // Exibe uma mensagem no terminal para avisar que está tudo funcionando
  console.log("🚀 Servidor Nativo rodando em: http://localhost:3000");
});
```

Comando para iniciar o servidor:

No terminal do VSCODE

node server.js